```
array(2) {
  [0]=>
  string(3) "Bob"
  [1]=>
  string(5) "Alice"
}
```

The `array()` language construct creates a new array, as does `[]`. The example above shows initialization with and without values, `[]` is an empty array, `['Bob', 'Alice']` is an array with two string elements.

The data values inside an array are usually referred to as *elements*. Array elements can be scalar or complex types. When an array is created in this way, PHP keeps track of the elements by indexing them. Each element is referenced by an integer value; the values are sequential and begin at zero.

Accessing Elements

To access the contents of an array, a special value called a *key* is required. An array key is a value that corresponds to the position of an element within the array. For the simple arrays we've seen so far, keys are simply sequential decimal integers corresponding to the position of the array element (starting at zero). Integer keys are also known as *indexes* (or *indices*). The key (or index) is then placed inside square brackets immediately following the array variable, for example, `$myArr[0]`. (Note that prior to PHP 8 it was also possible to access array elements with braces like this: `$myArr{0}`; this was deprecated in PHP 7.4 and is removed from the language from 8.0 onwards.)

Without diving too deep into zvals and gory PHP internals, the `$myArr` variable is a reference to an array, and we can think of the key notation (the bit inside the square brackets immediately following `$myArr`) as the reference to a specific element within that array.

In the output of the previous example, did you notice that Bob and Alice had numbers associated with them when the array was output with `var_dump`? Here's a reminder:

```
  [0]=>
  string(3) "Bob"
```